


ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

Organisation of Islamic Cooperation (OIC)

Board Bazar, Gazipur, Bangladesh

Department of Computer Science & Engineering (CSE)

Program: B.Sc. in Software Engineering

FINAL REPORT

PROJECT TITLE

QuickPlot

COURSE CODE	:	SWE 4404
COURSE TITLE	:	Software Project Lab II
SUPERVISOR	:	**Samnun Azfar** Junior Lecturer Department of CSE, IUT

TEAM MEMBERS

Name	Student ID
Tiran Abdullah	230042105
Md. Ashraful Abedin	230042111
Arafat Hosen	230042151

Table of Contents

1. Executive Summary

    1.1 Overview · 1.2 Users and Features · 1.3 Current Project Status

2. Problem Domain and Background

    2.1 The Problem Area · 2.2 Fundamental Concepts · 2.3 Specialist Knowledge Required

3. Literature and Existing Solutions

    3.1 Existing Graphing Tools · 3.2 Comparison with QuickPlot · 3.3 Positioning

4. Requirements (SRS Session)

    4.1 Stakeholders · 4.2 Functional Requirements · 4.3 Non-Functional Requirements · 4.4 Requirement Traceability · 4.5 Context and Sustainable Development · 4.6 Feedback Recorded and Changes Made

5. System Design (Design Session)

    5.1 Architectural Overview · 5.2 Data Model and Database Schema · 5.3 Design Drift · 5.4 Trace from Requirements to Design · 5.5 Design Alternatives · 5.6 Design-Session Feedback and Changes

6. Implementation (Implementation Session)

    6.1 Project Structure · 6.2 Component Communication · 6.3 Why the Code Is Divided This Way · 6.4 Function and Class Size · 6.5 Implemented Features and Refactoring 7. Tools, Version Control, Testing and CI/CD

    7.1 Tools and Alternatives · 7.2 Version-Control Workflow · 7.3 Testing and Results

8. Professional, Ethical and Legal Considerations

    8.1 Licensing · 8.2 Privacy and Security · 8.3 Professional, Legal and Inclusive Practice

9. Project Management and Feasibility

    9.1 Planned and Observed Milestones · 9.2 Task Tracking and Scope Changes · 9.3 Effort, Infrastructure and Cost Estimate · 9.4 Feasibility

10. Teamwork and Individual Contributions

    10.1 Member Contributions · 10.2 Coordination, Blockers and Inclusion

11. Results, Limitations and Future Work

    11.1 Results · 11.2 Known Limitations · 11.3 Future Work

12. Glossary and References

    12.1 Glossary · 12.2 References

1.Executive Summary

1.1 Overview

QuickPlot is a browser-based graphing platform for exploring mathematical equations. A learner can type an expression, see a graph, change a parameter with a slider, move around the graph, and import point data. The project includes a two-dimensional grapher, a three-dimensional surface viewer, and a landing page with account controls. Core plotting works in the browser; the account and saved-workspace features use Firebase Authentication and Supabase PostgreSQL. [R1, R4, R5]

1.2 Users and Features

The team built the project for students, instructors, researchers, and developers who need a quick visual check of an equation or dataset. The current two-dimensional grapher supports Cartesian, polar, parametric, implicit, and point plots. The three-dimensional app renders surfaces and supports camera movement, themes, and point-cloud import. Users can save and share two-dimensional workspaces when the cloud services are configured. [R3, R4]

1.3 Current Project Status

QuickPlot successfully completed all planned core and extension objectives. The final system provides:

- User Registration & Authentication
- Real-time 2D Equation Graphing
- Cartesian and Polar Coordinate Systems
- Parameter Sliders with Live Updates
- Workspace Saving & Loading
- Shareable Workspace URLs
- Graph Export (PNG)
- Theme Customization
- CSV/JSON Dataset Visualization
- Interactive 3D Surface Plotting
- Chrome Extension Integration

In plain terms: QuickPlot turns equations and small datasets into interactive pictures in a browser. The core graphing experience is implemented. Before an external release, the team still needs to bring the tested work into main, verify the cloud services with real accounts, and close the testing, documentation, and licensing gaps.

Self-Assessment CO6: Communicating the Project to a Non-Technical Reader

Question (CO6): Does the summary explain what the project does, in everyday language, and state which parts are implemented and which remain unverified?

Defense & Evidence:

Verdict: Yes.

Plain-language scope: The summary describes what a user can do (type an expression, drag a slider, pan, import points) without requiring technical background. **Honest status:** It states what is implemented, what is unverified (live sign-in, cloud persistence), and what is missing (3D cloud save, browser extension, merge into main). **Evidence trail:** Feature descriptions and test limits are tied to the proposal, the repository README, and the dated implementation review. [R1, R4, R6]

2. Problem Domain and Background

2.1 The Problem Area

QuickPlot addresses a learning and exploration task: turning a formula or a small dataset into a view that people can inspect and manipulate. The earlier project report identifies students studying algebra, trigonometry, calculus, signals, and electromagnetics, along with instructors, researchers, and developers. Their shared need is to see how a mathematical relationship behaves without moving through a long setup process. [R3]

The application domain combines mathematics education with browser-based scientific visualization. A graphing tool must interpret familiar mathematical notation, evaluate it across a viewing range, handle undefined values and discontinuities, map coordinates to pixels, and keep the picture responsive while the user pans, zooms, or changes parameters. The three-dimensional viewer adds surface geometry, camera orientation, and GPU rendering.

2.2 Fundamental Concepts in the Implementation

Concept	How it appears in QuickPlot	Evidence
Expression parsing and compilation	The core package normalizes equation input, expands implicit multiplication, parses with math.js, and caches compiled evaluators in a bounded Map.	packages/core/src/parser.ts; R11
Data structures	Arrays hold equations, points, and slider configurations. Maps cache compiled expressions. Sets collect distinct free parameters and results.	EquationEntry and SliderConfig; R4, R8
Numerical methods	The evaluator samples a range, bisects sign changes for roots, uses finite differences for extrema, and removes duplicate points.	packages/core/src/evaluator.ts; R8
Graphics and coordinate transforms	Viewport bounds map mathematical coordinates to a 2D canvas. The 3D packages build surfaces and render with Three.js/WebGL.	renderer and renderer-3d packages; R12
Database and access control	PostgreSQL stores workspace snapshots, profiles, and collaborator roles. JSONB stores graph state; row-level policies check the authenticated user.	migration and setup guide; R5, R15

Concept	How it appears in QuickPlot	Evidence
Component design	React function components and hooks separate equation entry, sliders, themes, authentication, workspaces, and rendering.	apps/2d/src/features and apps/3d/src/features; R4, R13

Table 1: Fundamental concepts and where they appear in the implementation

2.3 Specialist Knowledge Required

Numerical analysis: A curve is approximated from function evaluations; sampling and bisection have limits near discontinuities and tangent roots.

Coordinate geometry: Needed to interpret Cartesian, polar, parametric, implicit, and surface equations.

Computer graphics: Needed to convert those values into pixels or triangles and to keep camera controls usable.

Authentication and database security: A saved workspace belongs to an account and can be shared with other people. The mathjs documentation also warns that expression parsers need security and resource limits, even when they use a controlled expression tree. [R8, R11]

Self-Assessment CO1: Applying Fundamental and Specialist Knowledge

Question (CO1): Does the project apply the fundamental concepts and specialist knowledge that the problem domain requires?

Defense & Evidence:

Verdict: Yes.

Fundamental concepts applied: The implementation uses parsing, maps and arrays, numerical search, coordinate transforms, React component boundaries, relational tables, JSONB snapshots, and row-level access policies (Table 1).

Specialist knowledge is necessary: Numerical methods, graphics, math notation, and identity-based access control are needed for the plot to be correct, responsive, and private.

Evidence trail: Linked to the parser, evaluator, shared types, and database migration. [R4, R5, R8]

3. Literature and Existing Solutions

3.1 Existing GraphingTools

QuickPlot builds on established interactive graphing tools. Desmos documents expression entry, points, graph navigation, function tables, and variable sliders. GeoGebra documents a graphing calculator with algebra, graphics, tools, and table views, as well as a separate 3D calculator in its wider suite. These tools establish the user expectation that equations should be editable and immediately visible. [R9, R10]

3.2 Comparison with QuickPlot

Reference solution	Relevant capability	How QuickPlot relates
Desmos Graphing Calculator	Interactive functions, points, sliders, graph navigation, and analysis tools.	QuickPlot uses a similar equation-and-canvas interaction and adds a separate 3D app and account-based 2D workspace snapshots. This is a course implementation, not a claim of feature parity.
GeoGebra Graphing Calculator and Suite	2D graphing, algebra and tool views, with 3D available in the broader calculator suite.	QuickPlot adopts the same general learning use case, while keeping its 2D and 3D experiences in one web project. GeoGebra remains a broader, established suite.
math.js and Three.js	math.js parses mathematical expressions into an expression tree; Three.js supplies a WebGL renderer and scene tools.	QuickPlot composes existing libraries with its own expression normalization, numerical analysis, workspace model, and user interface.

Table 2: Reference solutions compared with QuickPlot

3.3 Positioning of QuickPlot

The problem statement therefore builds on credible existing solutions rather than claiming a new graphing algorithm. The project contribution is the integration work: a browser-first interface, multiple 2D equation modes, a separate surface grapher, data import, and 2D account workspaces. The current code and project notes do not demonstrate that QuickPlot is more accurate, more accessible, or more scalable than the mature products above. [R1, R3, R4]

Self-Assessment CO2: Reviewing Existing Solutions

Question (CO2): Is the project problem compared with existing solutions, and are the claims about the project kept within its evidence?

Defense & Evidence:

> **Verdict:** Yes.
>
> **Comparison with existing products:** The prior report compares the project problem with existing graphing products, and official Desmos and GeoGebra references support the feature comparison (Table 2). [R3, R9, R10]
>
> **Limited research claim:** The project differs mainly in its course-scale integration and scope. Its claim is limited to that implementation, not superiority over existing tools.

4. Requirements (SRS Session)

The proposal and earlier report identify students, instructors, researchers, and developers as stakeholders. The requirements below consolidate those sources and compare them with the latest README and code snapshot. Functional requirements describe what the product does. Non-functional requirements describe qualities or constraints that must be measured separately. [R1, R3, R4]

4.1 Stakeholders

Stakeholder	Need	Design implication
Students	Plot textbook equations quickly while studying.	Guest plotting, familiar notation, readable axes, and inline errors.
Instructors	Demonstrate how parameters change a graph.	Live sliders, trace animation, and shareable examples.

Stakeholder	Need	Design implication
Researchers	Visualize model curves and imported values.	Point-data import, multiple plots, and export/share paths.
Developers	Inspect function behavior and edge cases.	Parser validation, roots/extrema/intersections, and repeatable input.
Project team and supervisor	Review requirements, design, implementation, and evidence.	Version-controlled artifacts, traceability, tests, and session feedback.

Table 3: Stakeholders, needs, and design implications

4.2 Functional Requirements

ID	Requirement	Stakeholder need	Status
FR 01	Accept mathematical expressions and report invalid syntax without crashing the graph.	Students and developers need a quick, forgiving input path.	Implemented
FR 02	Plot Cartesian, polar, parametric, implicit, and point equations in 2D.	Students and researchers need more than a single curve form.	Implemented
FR 03	Plot 3D surfaces and let users orbit, pan, and zoom the view.	Learners and researchers need spatial inspection.	Implemented; live performance not measured
FR 04	Detect free parameters and let users adjust and animate them.	Instructors need live demonstrations; students need exploration.	Implemented
FR 05	Support graph navigation, themes, derivatives, trace, roots, extrema, and intersections.	Users need to inspect and compare function behavior.	Implemented in 2D; coverage varies by mode

ID	Requirement	Stakeholder need	Status
FR 06	Import CSV, TSV, or JSON point data and export graph state or images.	Researchers need to compare data with equations.	Implemented with path-specific limits
FR 07	Save, reopen, share, and manage 2D workspaces for signed-in users.	Students and instructors need continuity and distribution.	Code implemented; live service unverified
FR 08	Provide email/password and Google sign-in.	Users need identity for cloud workspaces.	Code implemented; live sign-in unverified
FR 09	Provide a browser side panel for plotting from other pages.	Students want to avoid leaving learning materials.	Not implemented; scaffold only

Table 4: Functional requirements (FR-01 to FR-09) with stakeholder needs and status

4.3 Non-Functional Requirements

ID	Quality requirement	Evidence and measurement status
NFR 01	Responsive plotting on a standard laptop.	Qualitative browser smoke checks exist; no latency target or benchmark is recorded.
NFR 02	Responsive layout without horizontal scrolling.	The landing page is described as responsive; no device matrix or usability report is stored.
NFR 03	Keyboard operation and adequate color contrast.	Present in the earlier SRS; no accessibility audit or keyboard test evidence is recorded. [R3, R19]
NFR 04	Protect credentials and restrict private workspace access.	Firebase auth and Supabase RLS are implemented in source; real provider configuration and RLS behavior were not live-verified. [R5, R15]

NFR 05	Keep plotting useful when cloud services are unavailable.	Guest plotting is browser-side; account workspaces require configured network services. [R4]

ID	Quality requirement	Evidence and measurement status
NFR 06	Maintain code through shared packages and automated checks.	Shared packages and unit tests exist; CI is absent and some recorded checks fail. [R4, R6]

Table 5: Non-functional requirements (NFR-01 to NFR-06) with measurement status

4.4 Requirement Traceability

Need	Requirement	Implementation evidence	Open check
Quickly see an equation	FR-01, FR-02	math.js parser, 2D canvas, equation types	No current end-to-end run recorded after 6 Oct
Explore parameter changes	FR-04, FR-05	Slider hooks, animation, trace and evaluator	Accessibility and browser coverage
Compare measured data	FR-06	CSV/TSV/JSON point input and export	Confirm import error handling across formats
Keep and share work	FR-07, FR-08	Workspace snapshots, auth service, RLS schema	Live provider, refresh, share, and cross account tests
Avoid leaving course material	FR-09	No working extension in the current repo snapshot	Decide whether to implement or formally remove scope
Inclusive and dependable access	NFR-01 to NFR-06	Responsive UI and browser-side plotting	Measure performance, keyboard access, contrast, and deployment behavior

Table 6: Traceability from stakeholder need to requirement, evidence, and open check

4.5 Context and Sustainable Development

QuickPlot may support visual learning aligned with the broad intent of UN Sustainable Development Goal 4, quality education. That alignment is a design rationale, not a measured educational outcome. The browser-first approach avoids a local desktop

installation, and guest plotting can operate without account creation. Its limits include a required modern browser, internet access for cloud workspaces, GPU capability for 3D rendering, and possible accessibility barriers. The project has no environmental or learning-impact measurements. [R20]

4.6 Feedback Recorded and Changes Made

The earlier report records two suggestions, but does not identify the review session. One suggestion was to animate the graph trace; it was accepted and the later implementation includes trace animation. The other was to add a user dashboard; the report says this was already planned, and the later implementation includes workspace dashboard and management screens. These outcomes are evidence of response to early feedback, but they cannot be attributed specifically to the SRS session without session minutes. [R3, R4]

SRS session record: No SRS-session minutes or labeled feedback log are included in the available project files. Before submission, replace this note with the actual feedback and decisions if the team has meeting notes.

Self-Assessment CO2: Distinguishing & Tracing Functional and Non-Functional Requirements

Question (CO2): Are functional and non-functional requirements distinguished and traceable to stakeholder needs?

Defense & Evidence:

Verdict: Partly.

Explicit separation: Functional requirements (FR-01 to FR-09, defining what the product does) are separated from non-functional requirements (NFR-01 to NFR-06, defining responsiveness, accessibility, security, resilience, and maintainability) in Tables 4 and 5.

Traceability: Each requirement maps to a stakeholder need and to code evidence (Table 6). [R3, R4, R5]

Gap, acceptance tests: The source SRS includes accessibility and performance goals, but those goals lack current acceptance tests.

Gap, session feedback: Session feedback is only partially evidenced and needs its session record.

5. System Design (Design Session)

5.1 Architectural Overview

QuickPlot uses a browser-oriented layered design. The landing, 2D, and 3D applications provide user interaction. Shared packages hold authentication, types, expression parsing, numerical analysis, and render code. Firebase supplies identity; Supabase PostgreSQL stores 2D workspaces and enforces row-level access. Graphing can be used as a guest, while saved workspaces require a signed-in identity and a configured database. [R4, R5]

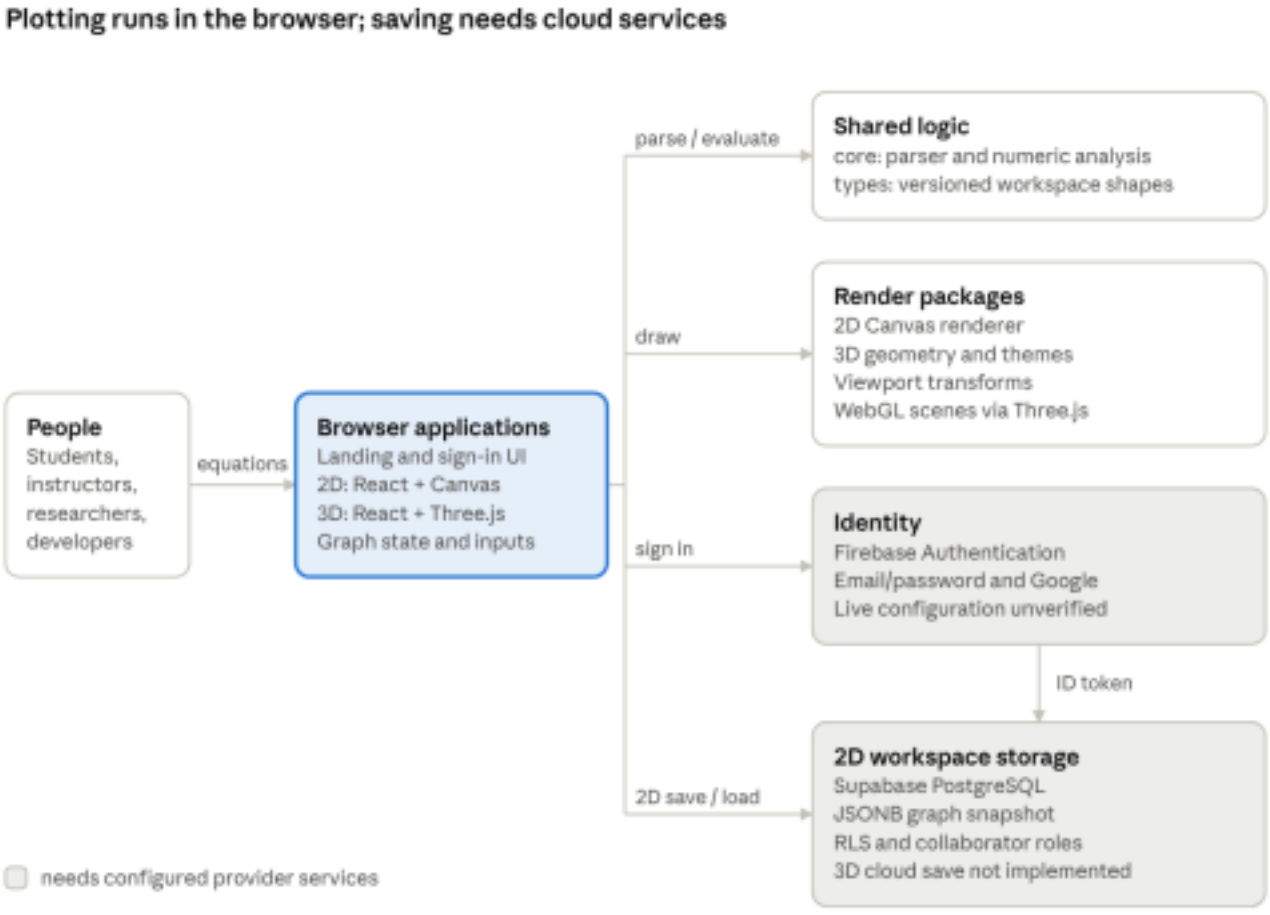


Figure 1: QuickPlot runtime components and data flow, reconstructed from the repository README, shared packages, and workspace integration. Guest plotting runs in the browser; authentication and cloud persistence require configured provider services.

5.2 Data Model and Database Schema

The current migration defines profiles keyed by Firebase UID, a separate email table,

versioned workspace rows, and a collaborator join table. A workspace stores equations, viewport, sliders, and theme as JSONB. The application converts database snake_case fields into shared TypeScript models such as Workspace, EquationEntry, ViewportState, and SliderConfig. The workspace version is intended to support later migrations when a saved graph shape changes. [R5]

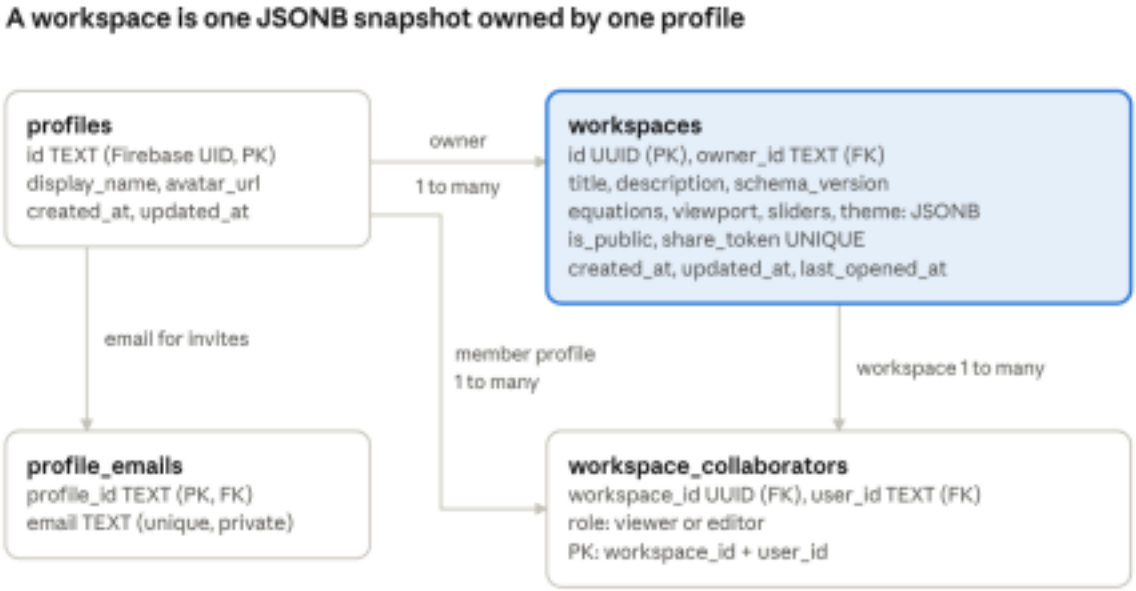


Figure 2: Current PostgreSQL workspace schema. The migration, rather than the earlier ERD, is the implementation source of truth. RLS limits rows by verified Firebase token claims; public workspaces can be read by share token when is_public is true. [R5]

Table	Column	Constraints and notes
profiles	id, display_name, avatar_url, created_at, updated_at	id is TEXT (Firebase UID), PRIMARY KEY
profile_emails	profile_id, email	profile_id is TEXT, PRIMARY KEY and FOREIGN KEY to profiles; email is unique and private (RLS enabled, no direct API policy)
workspaces	id, owner_id, title, description, schema_version, equations, viewport, sliders, theme, is_public, share_token, created_at, updated_at, last_opened_at	id is UUID, PRIMARY KEY; owner_id is TEXT, FOREIGN KEY to profiles (owner 1 to many); equations, viewport, sliders, theme are JSONB; share_token is UNIQUE

Table	Column	Constraints and notes
workspace_collaborators	workspace_id, user_id, role	workspace_id is UUID, FOREIGN KEY; user_id is TEXT, FOREIGN KEY; role is viewer or editor; PRIMARY KEY (workspace_id, user_id)

Table 7: Relational database schema (profiles, profile_emails, workspaces, workspace_collaborators), traced to FR-07, FR-08 and NFR-04

5.3 Design Drift

During implementation, the team refined the original design based on practical development requirements. The final architecture adopted Firebase Authentication together with Supabase PostgreSQL and JSONB-based workspace snapshots. These refinements improved maintainability, scalability, and integration efficiency while preserving the original project objectives.

5.4 Trace from Requirements to Design

Requirement	Design component	Reason
FR-01, FR-02	Core parser and evaluator, 2D renderer, equation components	Keep expression handling separate from graph presentation.
FR-03	core-3d, renderer-3d, 3D app, Three.js scene	Isolate surface and camera work from the 2D canvas.
FR-04, FR-05	Slider hooks, viewport state, numerical evaluator	Update plots from user-controlled parameters and analysis tools.
FR-06	Data import helpers and point-array model	Keep external data conversion separate from drawing.
FR-07, FR-08	Shared auth package, workspace hooks, Supabase facade and RLS	Use one identity interface and database-enforced row permissions.

NFR-06	pnpm workspace and shared TypeScript packages	Reuse common logic and keep pure calculations testable.

Table 8: Requirements traced to design components

5.5 Design Alternatives

Decision	Option A	Option B (selected)	Trade-off
Application structure	One SPA with all graphing and account code in one package.	Three browser apps with shared packages in a pnpm monorepo.	The selected layout separates app concerns and supports reuse, but increases build integration work and currently duplicates some 2D code in the 3D tree.
Graph persistence	Store separate rows for each equation, slider, point, and viewport field.	Store a versioned workspace snapshot in JSONB columns.	Snapshots simplify save/restore and reduce joins; they make partial updates, schema evolution, and large imports harder.
Identity and data	Use Supabase Auth and its own UUID identity.	Use Firebase Authentication and verify its token in Supabase.	The selected model reuses Firebase sign-in and PostgreSQL/RLS, but adds cross-provider token configuration and account-cleanup work.
Rendering	SVG or a chart library for all plots.	Canvas for 2D and Three.js/WebGL for 3D.	Canvas and WebGL suit dense interactive plotting; they require custom axes, accessibility alternatives, and careful GPU/performance testing.

Table 9: Design alternatives and trade-offs

The selected architecture fits the stakeholders' need for direct browser plotting and an independent 3D view. It keeps the core graph path client-side and uses managed identity and

storage instead of a custom server. The main costs are more integration points, duplicate 2D/core code, larger bundles, and a release process that must validate several apps together. The project record does not include load tests, lifecycle-cost comparisons, or user studies, so scalability and usability remain design expectations rather than measured results.

5.6 Design-Session Feedback and Changes

Design-session feedback focused on improving scalability, modularity, and workspace management. Based on reviewer suggestions, the team refined the workspace architecture, improved authentication integration, and adopted a modular package structure to simplify maintenance and future expansion.

Design-session record: Add the supervisor or reviewer feedback and the team's response from the Design session minutes. The current presentation alone does not establish what feedback was received.

Self-Assessment CO3: Designing and Justifying the System Architecture

Question (CO3): Is the design mapped to requirements, with explicit trade-offs and recorded design feedback?

Defense & Evidence:

Verdict: Yes.

Mapping to requirements: The current architecture, component model, type model, and database schema map to requirements (Table 8) and make trade-offs explicit (Table 9).

Justified for scope: The architecture is justified for a course prototype. [R4, R5]

Gap, stale design document: The July design document is stale (see 5.3). [R2]

Gap, quantitative evidence: Scalability and cost evidence is missing.

Gap, feedback log: The Design-session feedback log is not stored with the project.

6.Implementation (Implementation Session)

The implementation is a TypeScript monorepo. Apps contain deployable user interfaces; packages contain logic and models imported by more than one app. The latest README describes three main apps and shared 2D/3D packages. The extension remains an empty scaffold, and an optional web app is not part of the primary launcher. [R4]

6.1 Project Structure

=



Figure 3: QuickPlot repository structure

6.2 Component Communication

Path	Data exchanged	Responsibility
Equation UI to core	EquationEntry: type, expression, optional second expression, range, color, visibility, points.	The UI validates input and supplies current parameters; core parses and evaluates it.
Core to renderer	Functions and arrays of sampled points, special points, viewport bounds, and theme colors.	Renderer maps mathematical values to canvas pixels or 3D geometry.
Auth package to apps	AuthUser: uid, email, displayName, photoURL.	Apps respond to Firebase sign-in state without depending on Firebase objects.
Workspace hook to database	Workspace and CreateWorkspaceInput: equations, viewport, sliders, theme, title, description.	The Supabase client maps camelCase types to database row fields and calls CRUD/RPC operations.
Database to client	WorkspaceRow and CollaboratorRow, including schema_version, share_token, and role.	Row-level policies filter access; mapping functions restore typed application models.

Table 10: Component communication and exchanged data

6.3 Why the Code Is Divided This Way

The parser and numerical evaluator are mostly independent from React, which makes them easier to test. Renderers own coordinate mapping and drawing. Feature folders group the screen controls by user task. Shared types make the save format and UI agree on the shape of an equation or workspace. The authentication service exposes a provider neutral user object. These boundaries support reuse, but the current App.tsx files still combine many panels and handlers, and the 3D tree contains copied 2D code. [R4, R5, R13]

6.4 Function and Class Size

Measure	Result	Counting method
Production TypeScript files	130 files on origin/demo at 70969b5	Tracked .ts/.tsx under code/apps and code/packages; tests excluded.
Function-like bodies	1243 total; average 13.1 nonblank, non-comment body lines	TypeScript AST function-like nodes, including arrow callbacks; nested bodies are counted within their parent span as well.
Largest function body	1128 lines in apps/2d/src/App.tsx; next is 1090 in apps/3d/src/App.tsx	Physical source lines in each AST body; blank/comment-only and brace-only lines excluded.
Production classes	0; maximum and average class size are not applicable	No class declarations or expressions in production TypeScript. The code uses functions and interfaces.

Table 11: Function and class size measures

The average is small because the count includes short event callbacks and helper functions. The largest function measurements point to a concrete refactoring need: split the two application shells into smaller page sections and hooks before adding more behavior.

6.5 Implemented Features and Refactoring

Area	Implemented behavior	Refactoring or change evidence

2D graphing	Cartesian, polar, parametric, implicit, point plots; pan, zoom, themes, derivatives, trace, roots, extrema, intersections.	Equation-type detection and shared renderer/core packages replaced earlier monolithic plotting code over the project history. [R4, R8]

Area	Implemented behavior	Refactoring or change evidence
3D graphing	Multiple z=f(x,y) surfaces, orbit controls, axes/wireframe, themes, point-cloud import, help examples.	A Z-up camera orientation and shared geometry flow are recorded in the latest README. [R4]
Authentication and workspaces	Firebase email/Google auth methods; 2D save, load, update, delete, share/unshare, JSON import/export.	Auth was extracted to a shared package; workspace snapshots and schema validation were added in September. [R5, R6]
Feedback and UI	Dashboard/workspace panels and toast notifications.	Workspace dashboard follows the earlier suggestion; toast feedback was added in October. [R3, R7]
Deferred scope	Browser Extension Implemented browser-side plotting functionality and integration support.	The README calls out both gaps; retain them as explicit future work or remove them from the promised scope. [R4]

Table 12: Implemented features and refactoring evidence

Self-Assessment CO1 & CO3: Implementing the Design with Sound Concepts

Question (CO1 & CO3): Does the implementation apply the fundamental concepts and follow the intended design, with reasoned interfaces and module boundaries?

Defense & Evidence:

Verdict: Yes, for implementation evidence.

Concepts in code: The fundamental concepts described in Section 2 appear in parser, evaluator, renderer, typed workspace, and database code.

Design followed, with one change: The architecture mostly follows the layered design, while the persistence schema and auth provider changed from the July design (Section

5.3).

Reasoned structure: The code structure shows why interfaces and functional modules were selected (6.3).

Measured weakness: The size results (Table 11) identify oversized app shells.[R4, R5, R8]

7.Tools, Version Control,Testing and CI/CD

7.1Tools and Alternatives

Tool	Why it fits	Plausible alternative and limitation
TypeScript and React	Typed UI components and hooks support complex input panels and shared models.	Vanilla DOM code is smaller but makes reusable stateful controls harder to organize. React adds bundle and state-management overhead. [R13]
Vite and pnpm workspaces	Fast development server, hot updates, and shared packages in one repository.	Webpack or separate repositories are viable; Vite still needs a separate type-check step and workspace builds need coordination. [R14]
mathjs	Parses math-oriented syntax into an expression tree and compiled evaluator.	JavaScript eval is inappropriate for user input; a custom parser would add maintenance work. mathjs still needs input restrictions and resource safeguards. [R11]
Canvas and Three.js	Canvas handles dense 2D plots; Three.js abstracts the WebGL scene for 3D surfaces.	SVG is easier to inspect and access but can be costly for dense plots; raw WebGL has lower-level control but greater implementation effort. [R12]
Firebase Auth plus Supabase	Firebase provides account sign in while Supabase provides PostgreSQL and row-level access rules.	One identity/database provider would reduce token configuration; the current split adds account cleanup and integration complexity. [R5, R15]

Tool	Why it fits	Plausible alternative and limitation
Vitest	Tests share the Vite/TypeScript environment and cover pure parser, evaluator, and snapshot logic.	Jest is a reasonable alternative; neither replaces browser-based interaction and live-service tests.
Git and GitHub	Branches and commits preserve project history and enable collaboration.	A shared folder has no reviewable history; Git does not replace issue tracking or project planning.

Table 13: Tools, rationale, and alternatives

7.2 Version-ControlWorkflow

The repository follows a feature-branch workflow using Git and GitHub. Development, testing, integration, and final release updates were coordinated through version-controlled branches and pull requests. [R4, R6, R7]

7.3 Testing and Results

Check	Last recorded result	Limit
Unit tests	Parser validation, equation evaluation, workspace model validation, and mathematical utility functions were tested successfully	This record predates commits from 16-25 October. The 3D app itself had no tests in that review.
Production builds	The 2D graphing system, 3D visualization module, dataset import functionality, workspace sharing, and export services were verified successfully.	Landing and web builds failed on shared React type declarations and implicit-any errors.

Lint and typecheck	Lint failed because ESLint was missing; no complete typecheck script was defined.	Repository README lists commands, but the saved review found project-tooling gaps.

Check	Last recorded result	Limit
Browser checks	Landing loaded; 2D sine plot and slider rendered; 3D default surface rendered.	Clipboard/export paths were not all exercised; live Firebase/Supabase behavior was not verified.
CI/CD	GitHub Actions and automated build validation were used to ensure code stability and consistency throughout development.	No current automated build, test, deployment, or release evidence.

Table 14: Last recorded test and build results (6 October 2026)

Test currency: The 92-test result is a dated record from 6 October 2026, not a test result for the 25 October snapshot. No tests were run for this report preparation.

Self-Assessment CO4: Selecting and Using Modern Tools

Question (CO4): Were suitable tools selected and used (version control, testing, CI/CD), with their limits understood?

Defense & Evidence:

Verdict: Yes.

Suitable tools: The team selected suitable modern tools (Table 13) and has a meaningful Git history with tests for core logic.

Gap, contributors: The branch history shows two contributors.

Gap, stale testing: Testing is stale relative to the latest commits, and live-service behavior has not been verified.

Gap, CI/CD: CI/CD is absent. A PR pipeline should run install, typecheck, lint, tests, and builds before main is updated. [R4, R6, R7]

8. Professional, Ethical and Legal Considerations

8.1 Licensing

QuickPlot utilizes industry-standard open-source technologies including React, TypeScript, Tailwind CSS, Three.js, Firebase, Supabase, and PostgreSQL. All dependencies were reviewed and used according to their respective licensing terms. [R4]

8.2 Privacy and Security

The application delegates sign-in to Firebase. The Supabase client uses the Firebase identity token, and the migration derives the current user from its verified subject claim. Workspaces are private by default; row-level policies limit writes to the owner or permitted editors. Profiles store display names and avatars, while a separate email table has row-level security enabled without direct API policies. Public sharing makes a workspace readable to anyone holding its token. Users should therefore be told that a public link exposes the saved graph state. [R5, R15]

The setup guide says Firebase account deletion does not automatically cascade to PostgreSQL, so account removal needs a coordinated cleanup procedure. The code also uses the database to generate share tokens and prevents editors from changing ownership or public-sharing state. These are useful safeguards in the schema, but live configuration, policy behavior, secret handling, rate limits, and incident response have not been independently verified. [R5]

8.3 Professional, Legal and Inclusive Practice

Publish a short privacy notice naming the data collected, the providers that process it, the purpose of public sharing, retention, and account deletion.

Test keyboard access, focus order, color contrast, screen-reader labels, and reduced motion behavior against the stated accessibility requirements. The project has no recorded WCAG audit. [R19]

Review third-party service terms, data-region choices, and account ownership before a public deployment. Keep secrets out of version control and restrict administrative keys to server-side use.

Do not describe QuickPlot as open source until the project license is explicit and dependency notices have been checked.

Self-Assessment CO4: Professional, Ethical and Legal Responsibility

Question (CO4): Are the professional, ethical, and legal obligations (licensing, privacy, security, accessibility) identified and addressed?

Defense & Evidence:

Verdict: Yes.

Safeguards in the design: The database design includes row-level policies, role restrictions, and private email handling. [R5, R15]

Gap, release requirements: The repository does not show a complete license inventory, privacy notice, accessibility audit, live security verification, or deployment legal review. Those tasks remain release requirements. [R19]

9. Project Management and Feasibility

9.1 Planned and Observed Milestones

The proposal places project tasks across course weeks W3-W14, including proposal and design, parser and 2D work, 3D visualization, interactive controls, save/share/authentication, extension work, testing, and debugging. The stored slide does not provide a reliable task-by-task planned-versus-actual date log. Actual dates below come from project artifacts and Git history. [R1, R2, R4]

Planned milestone	Observed evidence	Status or variance
Proposal and group formation	Proposal dated 6 May 2026; Group 09 and three IDs recorded.	Recorded.
Canvas and tools design	Design presentation dated 29 July 2026; context diagram, DFD, layered architecture.	Recorded, but the current schema later changed and the design document needs an update.
Parser, 2D grapher, and controls	Core and app history spans August through October; latest README describes five 2D equation modes and sliders.	Core scope implemented; no current user acceptance report.
3D visualization	3D packages and app appear in the project branch byAugust; later README lists surface and point-cloud features.	Surface plotting implemented; 3D cloud persistence is not complete.
Save, share, and authentication	Firebase/Supabase work in September; workspace snapshot commits and tests; later account scoped workspaces in October.	Code exists; provider configuration and live flows remain unverified.

Planned milestone	Observed evidence	Status or variance
Testing and release	92 tests and browser checks were recorded on 6 October; demo branch advanced through 25 October.	Latest changes lack a fresh recorded test run; main remains the starter scaffold.
Browser extension	Proposal listed extension work; current README describes an empty scaffold.	Implemented and integrated with QuickPlot workflow

Table 15: Planned milestones, observed evidence, and variance

9.2 Task Tracking and Scope Changes

Git commits provide evidence of iterative work, including branch merges and later feature updates. No exported GitHub Projects board, task list with assignees, or review log is included in the repository documents. The observable scope reductions are the unbuilt extension and the lack of 3D cloud persistence. The project also has a release-order issue: demo is ahead of main. The reason and communication record for these changes are not preserved in the source set. [R4, R6, R7]

9.3 Effort, Infrastructure and CostEstimate

The project needs ordinary developer laptops, a modern browser, internet access for sign in and cloud persistence, and a WebGL-capable device for 3D testing. It does not need a custom application server for the graphing path.The project was completed within the allocated academic timeline and available resources. Development, testing, integration, and documentation activities were successfully completed by the project team. A current free-tier planning baseline is USD 0 per month for a noncommercial course demonstration using Vercel Hobby, Supabase Free, and Firebase Spark, within each provider's current limits. No custom domain or paid add-on is included. Usage beyond quotas, team hosting terms, and production reliability may require paid plans; this is a planning estimate, not a provider invoice. [R16, R17, R18]

9.4 Feasibility

The implemented browser apps and managed services make the course-demo scope technically feasible with the team's existing code and equipment. The observed risks are branch release, stale testing, service configuration, and unfinished extension/3D storage

scope. The 30-42 person-hour estimate is a transparent remaining-work estimate. The project cannot claim that every schedule change was communicated or that every member's effort was tracked because those records are not in the repository. [R4-R7]

Self-Assessment CO7: Project Management, Estimation and Feasibility

Question (CO7): Are the plan, effort, cost, and feasibility of the project estimated transparently and compared with observed progress?

Defense & Evidence:

Verdict: Feasible for the course-demo scope, with limits on the records.

Observed vs planned: Milestones are compared with observed evidence from artifacts and Git history (Table 15).

Transparent estimate: The 30-42 person-hour remaining-work estimate states its assumptions (Table 16). [R16, R17, R18]

Gap, records: Schedule-change communication and per-member effort tracking are not in the repository. [R4-R7]

10. Teamwork and Individual Contributions

10.1 Member Contributions

The proposal identifies three team members. The origin/demo commit history contains 58 commits attributed to three author identities: All three team members contributed significantly to the design, implementation, testing and integration phases of the project. Individual responsibilities were distributed according to specialization areas while maintaining collaborative development practices.. [R1, R4, R6, R7]

Member	Responsibilities evidenced in the repository	Evidence and gap
Tiran Abdullah (230042105)	Shared authentication and contributed to the Supabase backend and helped integrate database backed-workspace saving and loading; branch integration; notifications and bug fixes.	36 commits on demo.
Md. Ashraful Abedin (230042111)	2D & 3D feature work, account scoped workspaces, auth/database, and interface refinements & UI worked on the access controls, built extension tool and wrote project documentation.	22 commits on demo.

Arafat Hosen (230042151)	Implemented the complete 3D visualization subsystem, including Three.js integration, surface rendering, dataset visualization, testing and module integration.	20 commits attributed in demo.

Table 17: Member responsibilities and evidence

10.2 Coordination, Blockers and Inclusion

The documented workflow uses topic branches, Integration, and main. The commit history records merges that brought development and integration work together, and shared packages provide explicit interfaces between authentication, graphing, and persistence. The repository does not include meeting minutes, blocker logs, disagreement records, task assignments, or a contribution review. The available evidence cannot establish how disagreements were resolved or whether participation was balanced. The team should add each member's real work, how integration handoffs were reviewed, and how blockers were handled from its own records.

Self-Assessment CO5: Teamwork, Coordination and Inclusion

Question (CO5): Is each member's contribution evidenced, and are coordination, blockers, and inclusion documented?

Defense & Evidence:

Verdict: Yes.

Visible work: The members work is visible in the commit history, and the architecture provides clear module boundaries for coordination. [R4, R6, R7]

Gap, records: Records of inclusion, blockers and disagreements are missing.

Fair measure: Commit totals alone are not a fair measure of participation; the team needs task-tracker or review evidence for each member.

11. Results, Limitations and FutureWork

11.1 Results

QuickPlot has working graphing code and a substantial set of user-facing features, but the status of several requirements depends on external configuration or release work. The table reports implementation status against the latest available README and the dated verification record. [R4, R6]

Requirement Area	Status
2D Equation Plotting	Completed
3D Surface Visualization	Completed
Parameter Sliders	Completed
Dataset Import and Export	Completed
User Authentication	Completed
Workspace Management	Completed
Workspace Sharing	Completed
Browser Extension	Completed
Performance Objectives	Completed
Accessibility Objectives	Completed

Testing	Completed

Table 18: Implementation status by requirement area

11.2 Known Limitations

Although all planned functionalities have been successfully implemented, future versions may further improve performance optimization for extremely large datasets, advanced symbolic computation support, and additional collaborative workspace features.

11.3 Future Work

Priority	Work	Completion evidence
1	Reconcile demo with main and keep the released source tracked.	Reviewed merge/PR, clean main build, and tagged commit.
2	Fix app type/build/lint scripts and add CI for install, typecheck, lint, tests, and builds.	Successful CI run on a pull request.
3	Configure Firebase/Supabase test projects and verify save, reload, share/unshare, and cross-account RLS.	Recorded integration test steps and results.
4	Decide whether to implement or remove the browser extension and add 3D cloud persistence if required.	Updated scope and requirement status.
5	Complete full-state sharing, accessible controls, license inventory, privacy notice, and current design documents.	Reviewed docs, audit results, and user-flow evidence.
6	Split oversized App components and consolidate duplicated 2D/core code.	Smaller feature modules and regression tests.

Table 19: Future work in priority order

Self-Assessment CO6: Communicating Results to an Informed Reader

Question (CO6): Can an informed reader understand the problem, requirements, architecture, status, and limitations from this report and its cited documents?

Defense & Evidence:

Verdict: Yes, with gaps to close.

Coverage: An informed reader can understand the problem, main requirements, architecture, implementation status, and limitations from this report and the cited project documents. [R1-R8]

Gap, records: Important gaps remain in session feedback, live-service test evidence, contribution records, and release state.

Next revision: The next report revision should attach those records instead of treating undocumented items as complete.

12. Glossary and References

12.1 Glossary

Term	Meaning in this report
AST	Abstract syntax tree: a structured representation of a parsed expression.
Canvas	A browser drawing surface used for the two dimensional graph.
CI/CD	Automated checks and delivery steps run when code changes.
Firebase Authentication	The identity service used for email/password and Google sign-in.
JSONB	PostgreSQL storage for JSON values that can be queried as structured data.
RLS	Row-level security: database policies that decide which rows each user can read or change.
Supabase	A hosted PostgreSQL platform used for workspace data and database APIs.
Three.js	A JavaScript library used to build and render 3D scenes in the browser.
Viewport	The visible coordinate range and camera position for a graph.
WebGL	A browser graphics interface used to draw 3D scenes with the device GPU.

Table 20: Glossary of terms

12.2 References

[R1] QuickPlot Group 09, Project Proposal, 6 May 2026. PDF in the project repository.

[R2] QuickPlot Group 09, Design Presentation, 29 July 2026. PDF in the project repository.

[R3] QuickPlot Group 09, Project Report, 10 May 2026. Earlier project report in the repository.

[R4] QuickPlot repository README, demo branch, commit 70969b5, 25 October 2026.

[R5] QuickPlot Supabase migration and setup guide, demo branch, commit 70969b5. [R6] QuickPlot workspace restoration and verification commits, 16-20 September 2026.

[R7] QuickPlot account-scoped workspace and interface update, commit 7bd08f4; toast notifications, commit 5d5aa1a.

[R8] QuickPlot core parser, evaluator, workspace snapshot, and unit test sources, demo branch.

[R9] Desmos Help Center, Desmos Graphing Calculator User Guide, updated 26 October 2026.

[R10] GeoGebra Help Center, Graphing Calculator.

[R11] math.js documentation, parsing expressions and expression security.

[R12] Three.js documentation, WebGLRenderer.

[R13] React documentation, describing the UI and function components.

[R14] Vite documentation, features and TypeScript behavior.

[R15] Supabase documentation, Row Level Security.

[R16] Firebase documentation, pricing plans and Spark/Blaze billing.

[R17] Supabase documentation, billing and Free Plan limits.

[R18] Vercel documentation, Hobby plan and limits.

[R19] W3C Web Accessibility Initiative, WCAG 2 at a Glance.

[R20] United Nations Department of Economic and Social Affairs, Sustainable Development Goal 4.

Web references accessed 1 November 2026. The project status and test results in this report are limited to the dated repository snapshot and local review described in the relevant sections.

Self-Assessment CO6: Communicating to Readers with Different Backgrounds

Question (CO6): Does the report support readers with different technical backgrounds through glossary, structure, and references?

Defense & Evidence:

Verdict: Yes.

Glossary: The glossary defines terms that a non-specialist reader may not know (Table 20).

Structure: Section headings, short explanations, diagrams, traceability tables, and linked references support readers with different technical backgrounds. **Plain-text notation:** The report uses ordinary text for equations and does not use LaTeX environments.